🚗 ADAS & AD Portfolio

Vijay Priyan Ayyaru

📧Gmail | 🔗LinkedIn | 🐙 Github

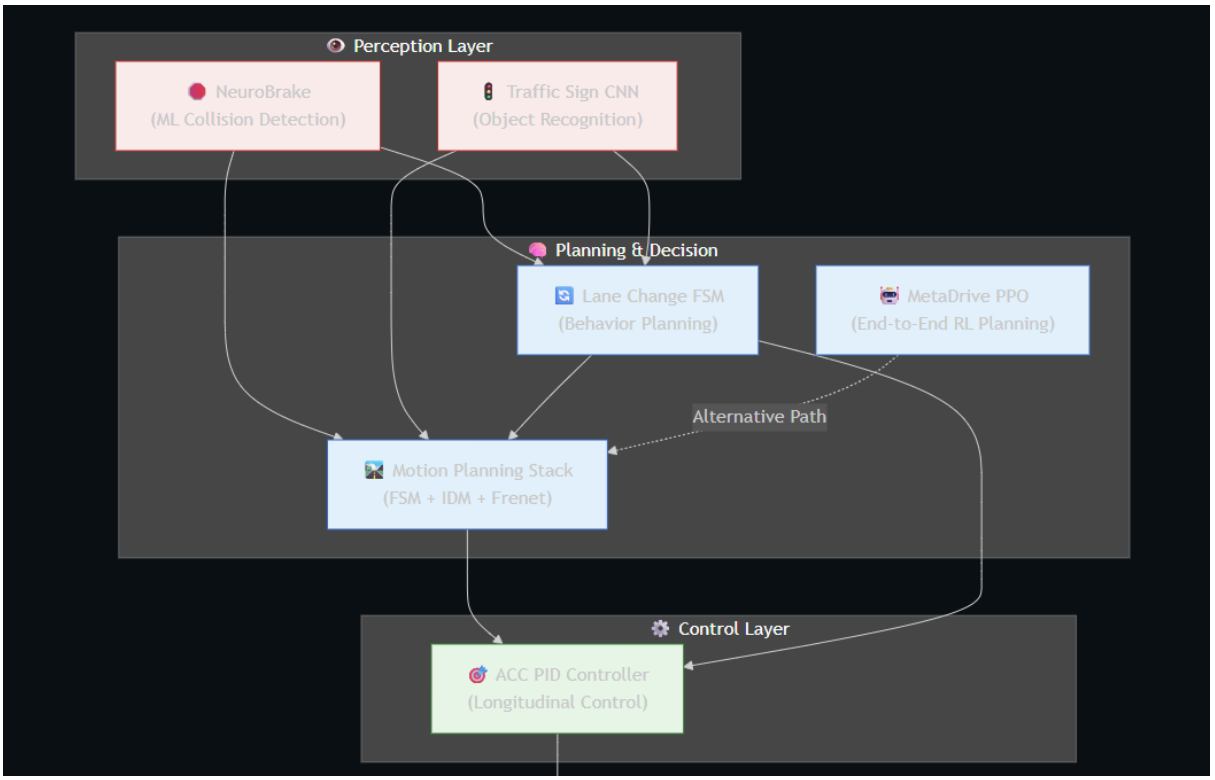
6 End-to-End Projects: Perception, Planning, Control & Deep Learning

📊 Portfolio Overview

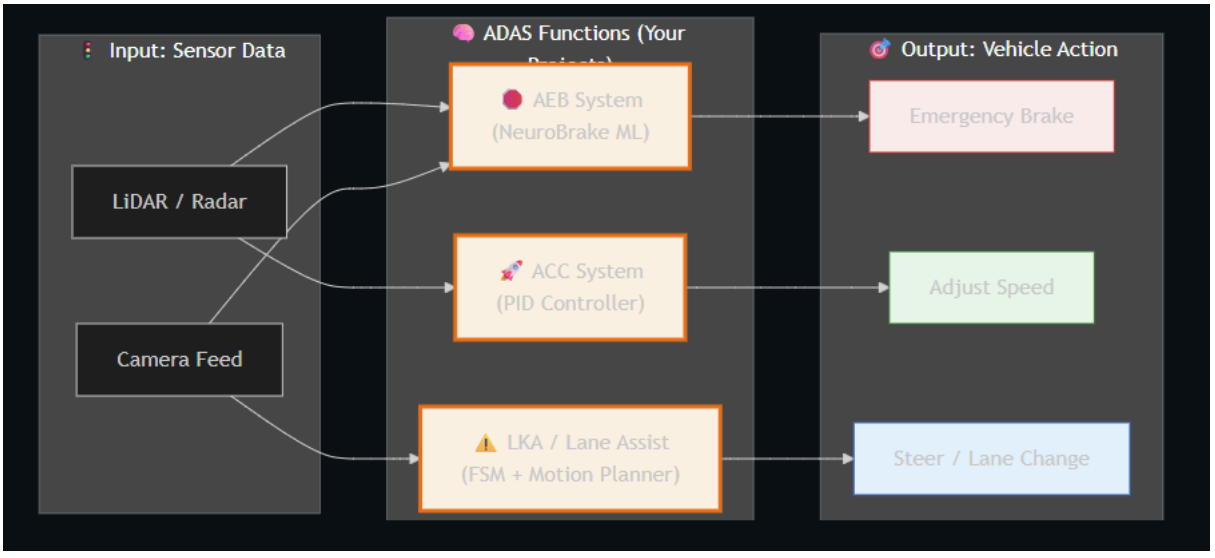
I have built **6 complete autonomous driving systems**, each tackling a different layer of the AV stack. This demonstrates **breadth** across all domains and **depth** in multiple specializations.

#	Project	Domain	Key Achievement
1	**NeuroBrake (ML-AEB)**	Safety / Perception	Neural net for collision prediction
2	**Adaptive Cruise Control (PID)**	Control Systems	Smooth distance keeping
3	**Highway Lane Change (FSM)**	Behavior Planning	Traffic-aware lane decisions
4	**Highway Motion Planning Stack**	Motion Planning	FSM + IDM + Frenet Lattice
5	**MetaDrive PPO Planner**	Deep RL	87% reward on unseen scenarios
6	**Traffic Sign Classifier (CNN+CLIP)**	Computer Vision	93.4% accuracy on GTSRB

AD focused Stack:



ADAS focused Stack:



🛑 1. NeuroBrake: ML-Based Automatic Emergency Braking

Domain: Safety-Critical Perception

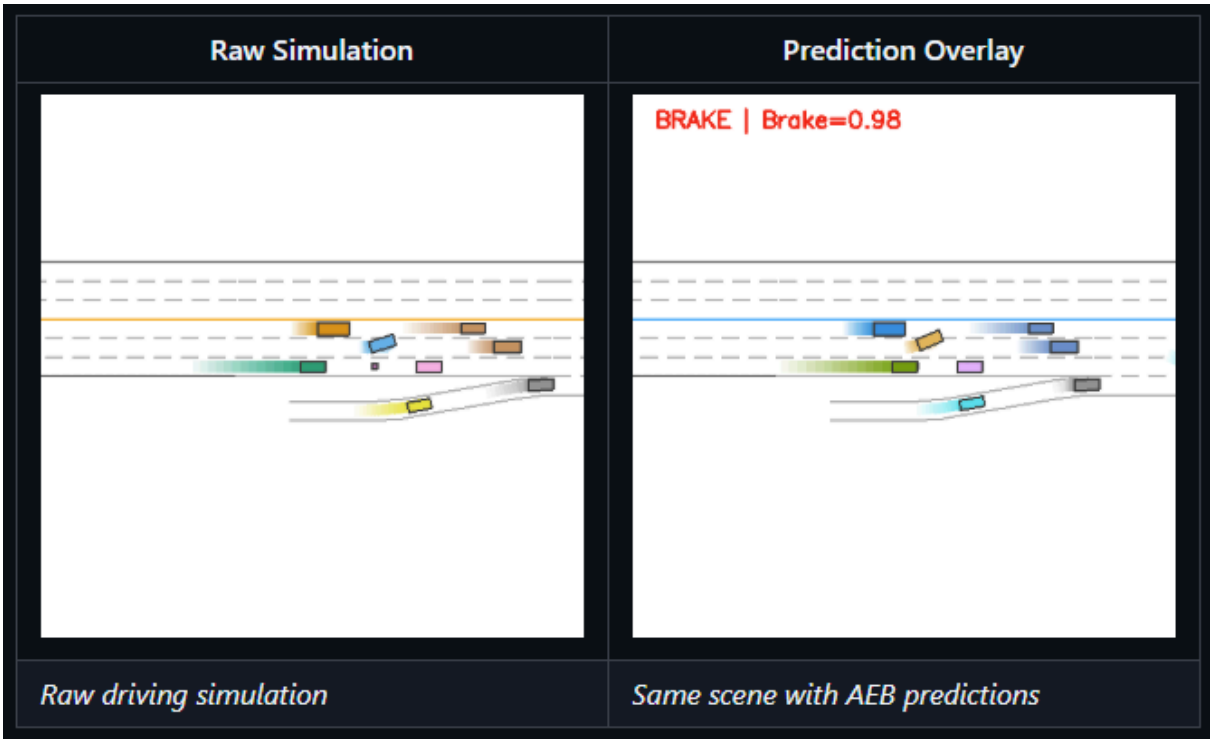
Goal: Predict collisions before they happen using sensor fusion.

Key Features:

- Neural network trained on simulated LiDAR + Radar data.
- Binary classification (Brake/No-Brake) with <50ms inference latency.
- Optimized for **zero false negatives** (safety-first design).

Tech Stack: `PyTorch` `Scikit-learn` `Sensor Fusion`

🔗 **Repository:** github.com/VPA2998/neurobrake-ml-aeb



🎯 2. Adaptive Cruise Control - PID Controller

Domain: Longitudinal Control

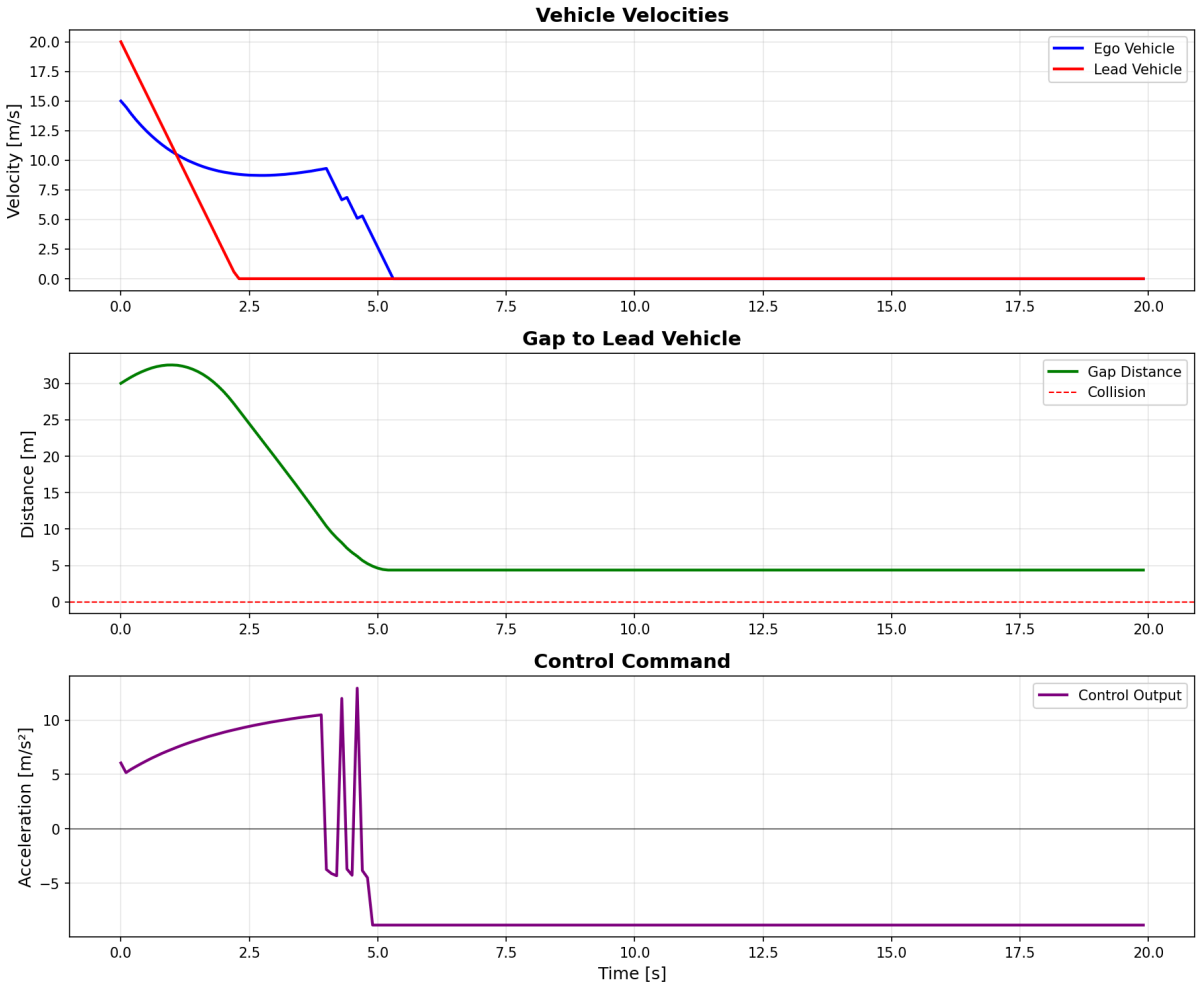
Goal: Maintain safe distance to lead vehicle with smooth acceleration.

Key Features:

- Classical **PID control** for throttle/brake commands.
- Gain tuning for comfort (low jerk) and safety (fast response).
- Simulated lead vehicle with varying speed profiles.

Tech Stack: `Python` `Control Theory` `Matplotlib`

🔗 **Repository:** github.com/VPA2998/adaptive-cruise-control-PID



🔄 3. Highway Lane Change - Finite State Machine

Domain: Behavior Planning

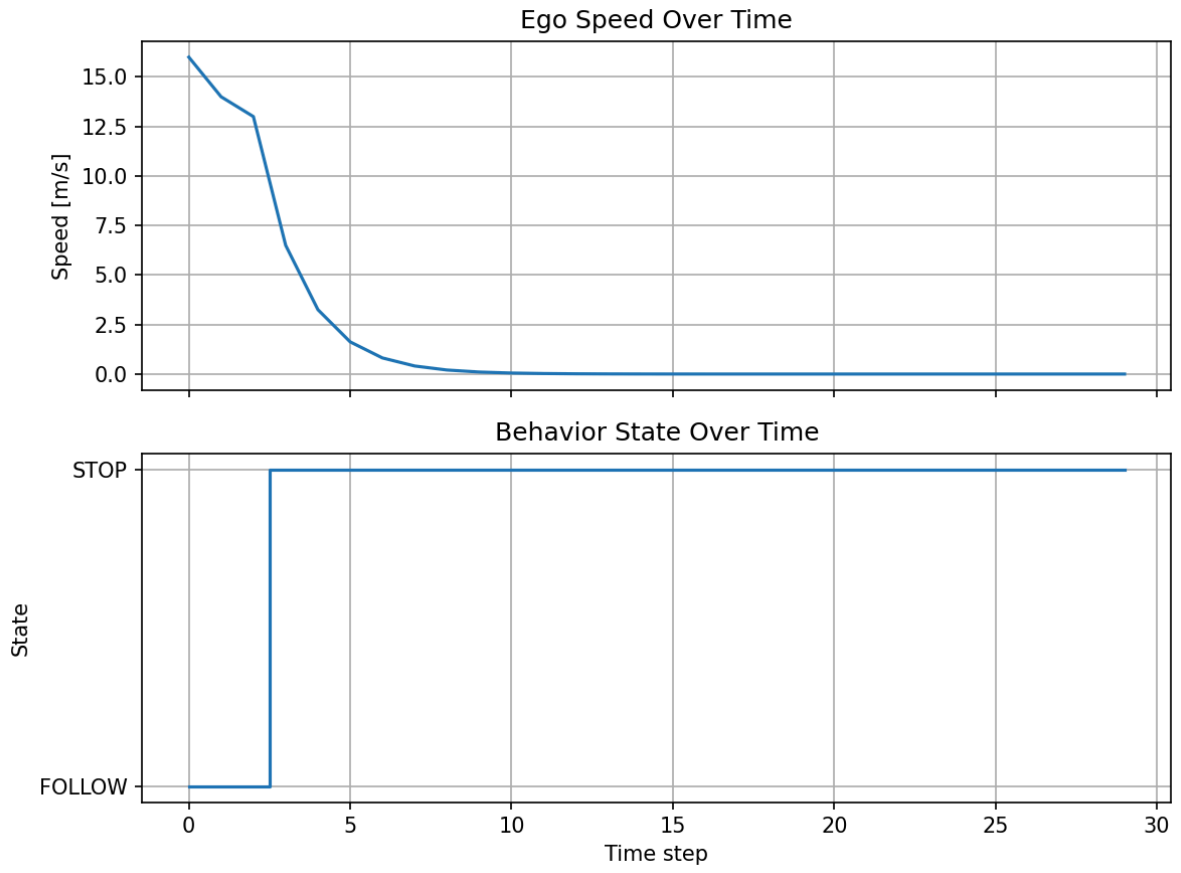
Goal: Safe lane change decisions in highway traffic.

Key Features:

- **5-state FSM:** Cruise, Follow, Lane-Change-Left, Lane-Change-Right, Stop.
- Gap assessment using **Time-to-Collision (TTC)** metrics.
- Hysteresis to prevent oscillating decisions.

Tech Stack: `Python` `FSM` `TTC Metrics`

🔗 **Repository:** github.com/VPA2998/highway-lane-change-behavior-planner-FSM



🛣️ 4. Highway Motion Planning Stack

Domain: Integrated Motion Planning

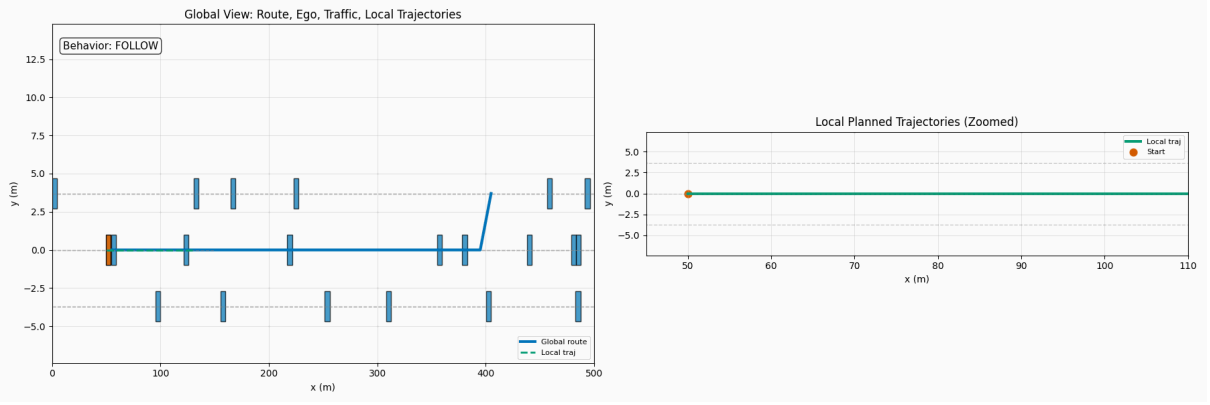
Goal: Complete highway driving with planning, control, and collision avoidance.

Key Features:

- **FSM** for high-level behavior + **IDM** for car-following.
- **Frenet Lattice** planner for smooth lateral trajectories.
- **Lanelet A**for global routing + **SAT** for collision detection.
- Modular `src/` architecture with 7 packages.

Tech Stack: `Python` `NetworkX` `Matplotlib` `Gradio`

🔗 **Repository:** github.com/VPA2998/highway-motion-planning-stack



🤖 5. MetaDrive PPO Highway Planner - Deep RL

Domain: End-to-End Reinforcement Learning

Goal: Learn highway driving without explicit rules.

Key Features:

- Trained **PPO** agent in MetaDrive simulator (100k timesteps).
- Observation: LiDAR + lane detectors (261-dim).
- **Results:** 87.2 ± 30.8 reward on unseen scenarios (7x better than random).

Tech Stack: `PyTorch` `Stable-Baselines3` `MetaDrive`

🔗 **Repository:** github.com/VPA2998/meta-drive-ppo-highway-planner

🎯 Scenario Coverage

We generated **9 demo GIFs** covering:

- **3 unseen seeds:** 1008, 1010, 1012
- **3 traffic densities:** 0.05 (light), 0.1 (medium), 0.2 (heavy)

📊 Performance Metrics

Metric	Value
Training Episodes	50 scenarios
Total Timesteps	100,000
Mean Reward (Train)	~100
Mean Reward (Eval, unseen)	87.2 ± 30.8
Generalization Gap	<13%
vs Random Agent	~700% better

🏆 Benchmark Comparison

We compared our trained PPO agent against a random action baseline on 5 unseen episodes:

Agent	Mean Reward	Std Dev	Success Rate
PPO (Ours)	**87.2**	30.8	~80%
Random Agent	12.5	5.2	~5%

Result: Our PPO agent achieves **~7x better performance** than random actions, demonstrating effective learning of highway driving behaviors.

🚦 6. Traffic Sign Classifier - CNN + CLIP

Domain: Computer Vision / Perception

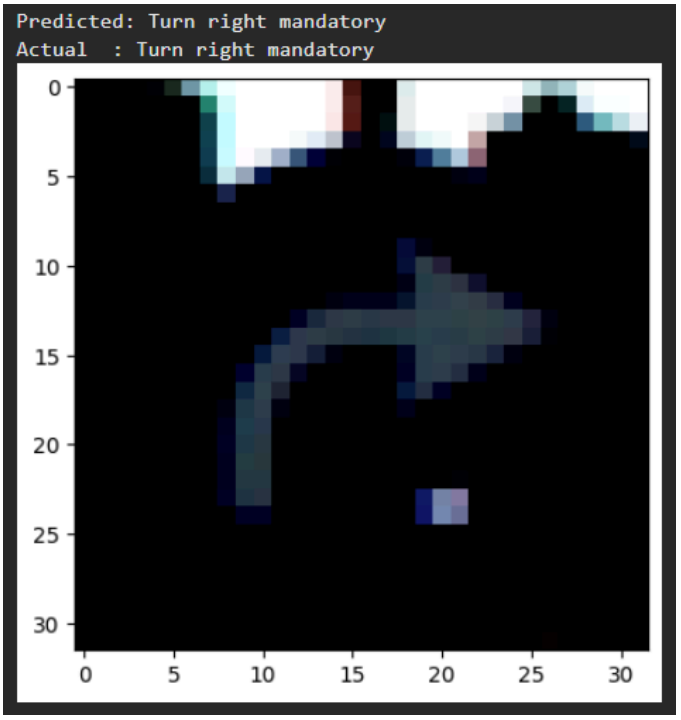
Goal: Recognize 43 traffic sign classes robustly.

Key Features:

- Custom CNN (2 Conv + 2 FC layers, ~549K params).
- **93.4% test accuracy** on GTSRB.
- Robustness tested on 7 corruption types.
- **CLIP** integration for human-readable explanations.

Tech Stack: `PyTorch` `torchvision` `Transformers (CLIP)` `Gradio`

🔗 **Repository:** github.com/VPA2998/traffic-sign-classifier-cnn



📧 Contact

Location: Kempten (Allgäu), Germany

📧Gmail | 🔗LinkedIn | 🐙 Github

Thank you for reviewing my portfolio.